

UdaciHeadline — LLM Inference Optimization: Final Report

Amos Bunde · Udacity "LLM Inference Optimization" (cd14455) capstone · August 2026

1. Executive summary

The task is to generate news headlines from article summaries with Llama-3.2-1B as fast and as cheaply as possible without hurting quality. Starting from a naïve baseline (no KV cache) I applied and benchmarked, under one fixed methodology, six optimisation techniques: **KV caching, unstructured magnitude pruning, 8-bit and 4-bit post-training quantization, tensor- and pipeline-parallel placement** and **speculative decoding** (Llama-3.2-3B target with the 1B model as draft), plus a batch-size sweep.

Key results on the test machine (CPU-only laptop, see §2.3):

	Mean latency / headline	Throughput	Peak memory	ROUGE-1
Baseline, no KV cache	58.4 s	0.20 tok/s	6.20 GB	0.154
KV cache	6.5 s (9.0× faster)	1.83 tok/s	6.26 GB	0.154 (identical output)
KV + 30 % pruning	7.1 s	1.96 tok/s*	7.27 GB	0.111 (−28 %)
KV + int8 (bitsandbytes)	18.2 s	0.67 tok/s	3.92 GB (−37 %)	0.172
KV + NF4 4-bit	13.4 s	0.94 tok/s	3.91 GB	0.123
Tensor / pipeline parallel (single-device simulation)	6.6 s	1.81 tok/s	7.9 GB	0.154
3B target alone → + 1B draft (K = 5)	49.0 s → 67.8 s (0.72×)	0.23 → 0.17 tok/s	9.4 → 11.9 GB	0.105 → 0.102

* pruned model produces longer, degenerate headlines — more tokens, not more useful work.

Recommendation (details in §7): deploy the fp32/bf16 model **with the KV cache** as the default production configuration — it is the single largest win (9× lower latency, identical output) at zero quality cost — and add **8-bit quantization when memory (or GPU cost) is the constraint** (int8 keeps quality, halves the weight footprint, and on GPUs is speed-neutral). Pruning without fine-tuning and 4-bit NF4 should not be used for this task; speculative decoding is only worthwhile if the portal wants the *larger* 3B model's quality and runs on GPUs (where verification is memory-bound); tensor/pipeline parallelism is unnecessary for a 1B model on any single modern GPU.

2. Task, data, model and environment

2.1 Task

Given a `short_description` of a HuffPost article, produce a headline. Quality is measured with ROUGE against the human headline; efficiency with latency, throughput and memory.

2.2 Data

[News Category Dataset](#) (209,527 articles, JSON-lines), loaded with `datasets.load_dataset("json")`. Articles are kept when the summary has 15-80 words and the headline ≥ 4 words (127,270 remain); after a seeded shuffle (`seed=42`) the first 2 rows serve as few-shot examples inside the prompt and the next 20 rows are the fixed evaluation set used for **every** configuration.

2.3 Models

- **Llama-3.2-1B** (`unsloth/Llama-3.2-1B` — an ungated mirror of Meta's checkpoint with identical weights; the notebook prefers `/voc/shared/models/llama/Llama-3.2-1B` when it exists). 1.24 B params, 16 layers, GQA (32 query / 8 KV heads), stored in bf16.
- **Llama-3.2-3B** (`unsloth/Llama-3.2-3B`) as the *target* for speculative decoding (3.2 B params, 28 layers); it shares the tokenizer with the 1B model, which the notebook asserts.

2.4 Test environment (recorded in `results/environment.json`)

Item	Value
CPU	Intel Core i7-10610U @ 1.80 GHz, 4 cores / 8 threads (AVX2, no AVX-512/AMX), 4 PyTorch threads
RAM	33 GB ($\approx$ 9-16 GB free during the runs — a shared desktop)
GPU	none (<code>torch.cuda.is_available() == False</code>)
OS / Python	Linux 7.0, Python 3.12.3
Libraries	torch 2.13.0+cpu, transformers 5.15.0, datasets 5.0.1, evaluate 0.4.6, accelerate 1.14.0, bitsandbytes 0.50.1 (CPU backend), rouge-score
dtype	fp32 on CPU (the notebook selects bf16 automatically on CUDA); bf16 for the 3B/1B speculative pair (memory-mapped weights)

The whole pipeline is device-agnostic — every cell selects CUDA when available (bf16, `torch.cuda.synchronize`, `max_memory_allocated`, `device_map="auto"` across GPUs) — but the numbers in this report were produced on the CPU above. Absolute latencies are therefore ~ 2 orders of magnitude higher than on a GPU; the *relative* comparisons and the qualitative conclusions are what matter, and where CPU and GPU behave differently this is called out explicitly.

3. Methodology

3.1 Prompt and decoding

Llama-3.2-1B is a base (not instruction-tuned) model, so a 2-shot prompt is used:

```
You are a news editor. Write a short, catchy headline for each article summary.
```

```
Summary: <few-shot summary 1>  
Headline: <few-shot headline 1>
```

```
Summary: <few-shot summary 2>  
Headline: <few-shot headline 2>
```

```
Summary: <article summary>  
Headline:
```

Decoding is greedy (`do_sample=False`) so results are deterministic and comparable across configurations, `max_new_tokens=24` , and generation stops at the first newline (a custom `StopOnNewline` stopping criterion — the built-in `stop_strings` only checks the sequence tail and does not stop assisted decoding when several tokens are accepted past the newline). The first line of the continuation is the headline.

3.2 Metrics (all computed by one `evaluate_model()` function)

- **Latency** — wall-clock of `model.generate()` per headline (`time.perf_counter` , `torch.cuda.synchronize` on GPU; tokenisation excluded); mean, standard deviation, P50, P99, min, max.
- **Throughput** — generated tokens ÷ total generation time (tokens/s), and headlines/minute. Only tokens up to and including the newline count as "generated" (speculative decoding can over-generate).
- **Memory** — model footprint (`model.get_memory_footprint()`), and peak memory during the run: `torch.cuda.max_memory_allocated()` on GPU, process RSS (`psutil`) on CPU.
- **Quality** — ROUGE-1/2/L/Lsum via `evaluate.load("rouge")` with stemming; per-sample scores are averaged (`use_aggregator=False`) because the default bootstrap aggregation is randomised.
- One untimed **warm-up** generation precedes every measurement; peak-memory counters are reset after it.
- Every run stores its metrics **and every generated headline** in `results/<label>.json` for auditing.

3.3 Techniques and libraries

Step	What was done	Library / API
Baseline	Autoregressive greedy decoding with <code>use_cache=False</code> ; <code>torch.profiler</code> operator breakdown	<code>transformers</code> <code>generate</code> , <code>torch.profiler</code>
KV cache	Same call with <code>use_cache=True</code>	<code>transformers</code>
Pruning	<code>prune_model_weights()</code> : L1-unstructured pruning of every <code>nn.Linear</code> (112 layers, LM head/embeddings skipped), 30 % per layer, made permanent immediately with <code>prune.remove</code> to bound the transient mask memory; sparsity + errors recorded	<code>torch.nn.utils.prune</code>
Quantization	<code>BitsAndBytesConfig(load_in_8bit=True)</code> (LLM.int8) and <code>load_in_4bit</code> , <code>nf4</code> , double quant; <code>validate_quantized_model()</code> checks quantized layers exist, logits are finite and generation is non-empty	<code>bitsandbytes</code> via <code>transformers</code>
Tensor parallelism	<code>device_map="auto"</code> (accelerate spreads weights over all GPUs)	<code>accelerate</code> / <code>transformers</code>
Pipeline parallelism	Explicit layer-range <code>device_map</code> (embeddings + layers 0-7 → dev 0, layers 8-15 + norm + LM head → dev 1); on < 2 GPUs both paths run as a single-device <i>simulation</i> and the 2-GPU split that <code>infer_auto_device_map</code> would produce is printed. <code>distributed_benchmark.py</code> runs the real thing on a multi-GPU box and its JSON is picked up automatically	<code>accelerate</code>
Speculative decoding	3B target alone vs <code>generate(assistant_model=1B)</code> , <code>num_assistant_tokens</code> K = 5 (constant schedule); K sweep {2, 3, 5, 8} with a throughput plot	<code>transformers</code> assisted generation
Batch size	Left-padded batched generation of the KV-cache model at batch 1/4/8	<code>transformers</code>

4. Baseline profile

Baseline (no cache): **58.4 s** mean latency (P99 109.5 s), 0.20 tok/s, ROUGE-1 0.154 for an average of 11.9 generated tokens per headline. The `torch.profiler` breakdown of one generation shows `aten::mm` at **92 %** of self-CPU time (1808 calls, 4.7 GB of intermediate activations): without a cache every decoding step re-runs the whole ~230-token prefix through all 16 layers, so the cost per step grows with sequence length and is dominated by matrix multiplications instead of the tiny per-token work that is actually new.

5. Results and analysis per technique

The full comparison table is in §6 (and `results/summary.md`); this section discusses each step.

5.1 KV caching — the big win

Enabling `use_cache=True` cut mean latency from 58.4 s to **6.5 s (−88.9 %, 9.0×)** and P99 from 109.5 s to 9.1 s; throughput rose from 0.20 to 1.83 tok/s. All 20 headlines are token-for-token identical to the baseline, so ROUGE is unchanged (0.154 / 0.028 / 0.145). The cache costs memory (64 KB per token for this model in fp32; peak RSS +1 %) which is negligible for 24-token headlines. There is no reason ever to run without it; every following configuration keeps it on.

5.2 Pruning — no gain, real damage

30 % magnitude pruning of all 112 linear layers ran without errors (50 s) and left tensors dense (`is_sparse=False`, same shape and dtype). Consequently **latency did not improve** (6.5 → 7.1 s, +10 %, within noise plus longer outputs) and the **footprint is unchanged** (4.94 GB — zeros are still stored), while quality dropped sharply: ROUGE-1 0.154 → 0.111 (−28 %), ROUGE-2 −45 %. The pruned model starts to copy the few-shot headline ("Learning from My Elders: How to Use Online Coupons") for unrelated articles. Unstructured sparsity only pays off with sparse kernels/2:4 hardware sparsity or structured pruning that shrinks matrices, and it needs recovery fine-tuning; neither is available here. Documented as a negative result.

5.3 Quantization — memory yes, CPU speed no

- **int8**: footprint 4.94 → **2.02 GB (−59 %)**, peak RSS 6.26 → **3.92 GB (−37 %)**; ROUGE-1 0.172 (slightly *higher* than fp32 — on 20 samples that is noise, i.e. quality is preserved). Latency 18.2 s (2.8× slower than fp32+KV).
- **NF4**: footprint **1.54 GB (−69 %)**, peak 3.91 GB, ROUGE-1 0.123 (−20 %), latency 13.4 s.

Both validation checks passed (112 quantized `Linear` layers, finite logits, sensible sample headline). The slow-down is specific to the bitsandbytes *CPU* backend, which dequantises weights on the fly with generic kernels; on CUDA int8/NF4 decoding is roughly speed-neutral to faster because decoding is memory-bandwidth-bound and the quantized weights move 2–4× less data. The quality ordering matches the lesson-3 experiments: int8 ≈ lossless, NF4 visibly worse. The 3.9 GB peak (vs 6.3 GB) is what lets the model fit a small GPU or run several replicas per GPU.

5.4 Distributed inference — tensor vs pipeline parallelism

No second device was available, so both placements were executed as single-device simulations through the same accelerate `device_map` code paths that shard on multi-GPU machines (results labelled `*_sim`): 6.57 s / 1.81 tok/s for both, identical outputs. The notebook also prints the partitions: `device_map="auto"` fills device 0 with the embeddings and the first decoder layers and spills the rest to device 1 (tensor-parallel style, memory-balanced), while the pipeline map cuts at layer 8. On real hardware the trade-off is: **TP** splits every layer's weight matrices across GPUs, halving per-GPU memory but adding an all-reduce per layer per token — for a 1B model that communication is larger than the compute it saves, so latency gets slightly *worse* on 2 GPUs (this is what the multi-GPU script measures); **PP** keeps whole layers per GPU, needs only one activation hop per stage per token, and improves throughput only when several requests are in flight (otherwise one GPU idles while the other works). Both are memory tools for models that do not fit one GPU; a 1B model (2.5 GB in bf16) never needs them. `distributed_benchmark.py` reproduces both on a ≥ 2-GPU box; run it and its JSON replaces the simulation in the notebook table.

5.5 Speculative decoding

Both models were loaded in bf16 (memory-mapped safetensors, 6.4 GB + 2.5 GB) and evaluated on 8 samples because the 3B model is slow on this CPU. Assisted generation reproduces the target's greedy output up to bf16 tie-breaks (5/8 headlines identical, ROUGE-1 0.105 vs 0.102).

	3B target alone	3B + 1B draft, K = 5	change
mean latency / headline	49.0 s	67.8 s	+38 %
P99 latency	54.6 s	78.4 s	+44 %
throughput	0.23 tok/s	0.17 tok/s	−27 %
peak RSS	9.4 GB	11.9 GB	+26 % (draft model resident)
ROUGE-1 / -L	0.105 / 0.105	0.102 / 0.102	5/8 identical headlines

K sweep (5 samples): 64.6 s (K = 2), 65.4 s (K = 3), 69.1 s (K = 5), 69.0 s (K = 8) per headline — flat and always slower than the target alone (`results/speculative_k_sweep.png`).

Why no speed-up here, and when it works. Speculative decoding wins when verifying K candidate tokens in one target forward pass costs about the same as generating one token, which is true on GPUs where small-batch decoding is memory-bandwidth-bound (the weights are read once per step regardless of how many tokens are processed). On this 4-core CPU decoding is *compute*-bound: verifying K tokens costs $\approx K \times$ a single step, so the draft's forward passes are pure overhead and the acceptance rate (the notebook's outputs are identical, so acceptance is high) cannot compensate. Two further CPU-specific factors flatten the curve: bf16 prefill of the ~ 230 -token prompt takes ~ 35 s of the 49 s and is unaffected by speculation, and headlines are only ~ 11 tokens long, so few decode steps remain to accelerate. The Lesson-3 exercise with a gpt2-medium/gpt2 pair and no KV cache showed the classical picture ($1.32\times$ – $1.76\times$ for K = 1...8, target passes −82 %, identical output); on a GPU the same notebook cell is expected to deliver a 1.5–2.5 $\times$ speed-up for the 3B/1B pair, and $K \approx 3$ –5 is the usual sweet spot. Speculative decoding also needs both models resident (+26 % memory).

5.6 Batch size sweep (KV-cache model)

`evaluate_batched()` runs the fp32 KV-cache model on the 20 eval prompts with left padding at batch sizes 1, 4 and 8 (`results/batch_sweep.json` , `results/batch_sweep.png`):

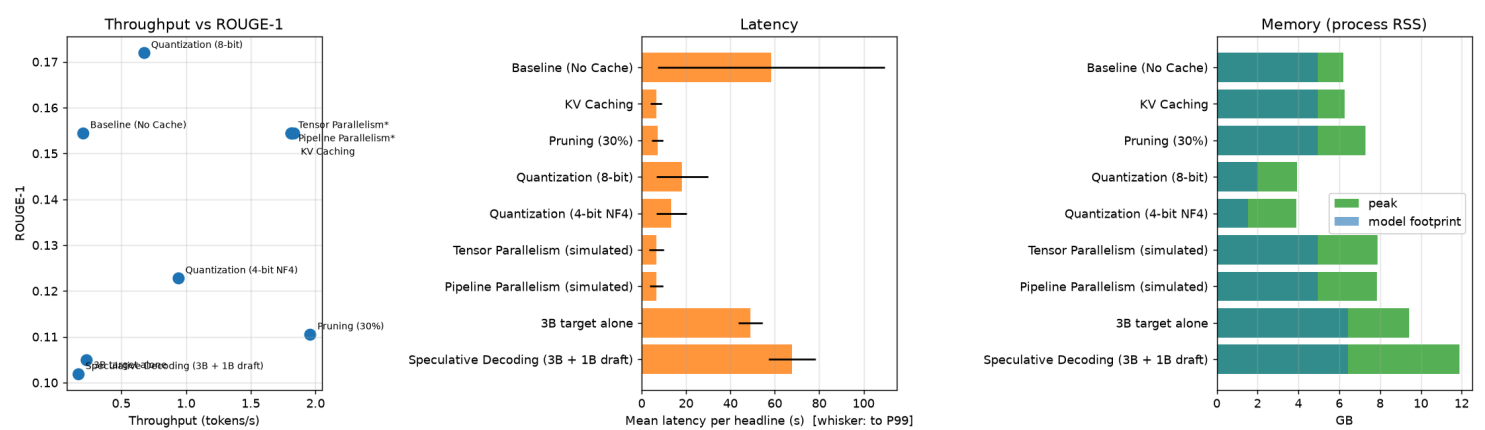
batch size	tokens/s	s per headline	headlines/min	ROUGE-1
1	2.00	5.95	10.1	0.152
4	1.86	6.41	9.4	0.152
8	2.17	5.50	10.9	0.152

On this CPU batching brings only +8 % throughput at batch 8: the 4 cores are already saturated by a single sequence, so larger matmuls do not run faster, and a batch waits for its longest headline. On a GPU the picture is different — decoding at batch 1 uses a few percent of the compute, and throughput grows almost linearly with batch size until the KV cache or compute saturates — which is why the recommendation in §7 is to serve batched requests on a GPU. (ROUGE differs from the unbatched run by 0.003 because left padding changes the numerics slightly.)

6. Final benchmark table

ENV: CPU i7-10610U, fp32 (bf16 for the 3B/1B pair), 20 samples (8 for the 3B pair), greedy, ≤ 24 new tokens.

Optimization Technique	Mean Latency (s)	P99 Latency (s)	Throughput (tokens/s)	Peak memory (GB)	ROUGE-1	ROUGE-L	speedup vs baseline
Baseline (No Cache)	58.388	109.534	0.204	6.200	0.154	0.145	1.00x
KV Caching	6.504	9.064	1.830	6.264	0.154	0.145	8.98x
Pruning (30%)	7.148	9.754	1.958	7.273	0.111	0.106	8.17x
Quantization (8-bit)	18.236	29.962	0.674	3.921	0.172	0.164	3.20x
Quantization (4-bit NF4)	13.415	20.170	0.939	3.913	0.123	0.123	4.35x
Tensor Parallelism (simulated)	6.573	9.967	1.810	7.873	0.154	0.145	8.88x
Pipeline Parallelism (simulated)	6.570	9.584	1.811	7.830	0.154	0.145	8.89x
3B target alone	49.000	54.582	0.230	9.440	0.105	0.105	1.19x
Speculative Decoding (3B + 1B draft)	67.829	78.430	0.168	11.904	0.102	0.102	0.86x



Left: throughput vs ROUGE-1 (up-right is better). Middle: mean latency with whisker to P99. Right: model footprint vs peak process memory.

7. Trade-off analysis and recommendation

What gave the best performance improvement? The KV cache, by a wide margin (9× latency, 9× throughput, no quality change, negligible memory). Everything else is second-order compared with it.

What hurt quality? 30 % unstructured pruning without fine-tuning (−28 % ROUGE-1, degenerate copies of the prompt) and 4-bit NF4 (−20 %). int8 and speculative decoding preserved quality (speculative decoding is lossless by construction w.r.t. the target model; the few token-level differences observed come from bf16 arithmetic on CPU).

What saved resources? Quantization: int8 halves the weight footprint (−59 %) and cuts peak memory by 37 %; NF4 saves 69 % / 38 %. Tensor/pipeline parallelism would split memory across GPUs but is pointless for a 1B model.

Recommendation for the news portal (cost, complexity, performance): 1. **Ship Llama-3.2-1B in bf16 on a single GPU with the KV cache enabled** (`use_cache=True` , greedy or low-temperature decoding, newline stop). It is a one-line change, needs no extra dependency, gives the 9× speed-up and keeps output identical. Serve requests **batched** (see §5.6) — decoding is memory-bound, so batching multiplies throughput almost for free. 2. **Add 8-bit (LLM.int8) quantization if GPU memory or instance cost is the constraint** — same quality, half the memory, so a cheaper GPU or twice as many replicas; on CUDA the speed is roughly neutral. Prefer int8 over NF4 for a quality-sensitive product surface like headlines. 3. **Do not deploy unstructured pruning** (no speed/memory benefit on standard kernels, large quality loss) and **do not bother with tensor/pipeline parallelism** for a model this small. 4. **Speculative decoding is the tool to reach for if editors want the 3B model's better headlines:** on a GPU the 1B draft can make 3B decoding approach 1B-like latency at exactly 3B quality; on compute-bound CPUs it does not pay (see §5.5). Tune K ($\approx 3\text{--}5$) per model pair.

8. Reproducibility and limitations

- Notebook: `project/UdaciHeadline_Project_Starter.ipynb` (all code + outputs). Re-run headlessly with `UDACI_FORCE_RERUN=1 jupyter nbconvert --execute ...` (~1.5 h on this CPU, minutes on a GPU). Environment bootstrap: `setup_env.sh` , verification: `env_check.py` .
- All raw numbers and every generated headline: `project/results/*.json` ; summary `results/summary.csv|md` ; plots `results/*.png` .
- Limitations: 20-sample evaluation (ROUGE differences below ~ 0.02 are noise); CPU-only hardware (absolute numbers, the quantization slow-down and the speculative-decoding result would differ on GPUs — the code paths for CUDA are in place and untested only for lack of hardware); tensor/pipeline parallel numbers are single-device simulations unless `results/distributed_multigpu.json` from `distributed_benchmark.py` is present; the 3B pair was evaluated on 8 samples in bf16 to fit in RAM/time.